DESIGN PATTERNS

Farah Younas

Table of Contents

CREATION PATTERNS 2

1. Factory Pattern 2

1.1. Simple Factory Design Pattern 2

1.2. Exercise — Food Ordering System 4

1.3. Factory Method Design Pattern 4

1.4. Exercise — Online Notification System 6

1.5. Abstract Factory Pattern 6

2. Singleton Pattern 7

2.1. Exercise — University Configuration Manager 9

3. Builder Pattern 9

3.1. Exercise — Build a Travel Package 13

STRUCTURAL PATTERNS 13

1. Decorator Pattern 13

1.1 Exercise — Car Customisation 14

2. Adapter Pattern 15

2.1 Combination 15

2.2 Multiple Inheritance 16

2.3 Exercise — Payment System 16

BEHAVIORAL PATTERNS 17

1. Observer Pattern 17

1.1 Exercise — Stock Price Monitor 19

CREATION PATTERNS

1. Factory Pattern

- The Factory Design Pattern is a way of creating objects without directly specifying which class should be used.
- The main benefit of the Factory Pattern is that it separates object creation from the code that uses the object.

1.1. Simple Factory Design Pattern

Imagine you have an online payment system.
A customer can pay using:

- Credit Card
- PayPal
- Bank Transfer

Without the Factory pattern, you might write:

```
payment_type = "paypal"

if payment_type == "credit_card":
    payment = CreditCard()
elif payment_type == "paypal":
    payment = PayPal()
elif payment_type == "bank":
    payment = BankTransfer()

payment.pay()
```

- The problem is that the code that uses the payment also has to know how to create each payment object.
- The Factory pattern moves this object-creation logic into a separate place.

Example I

Step 1: Create the classes
Step 2: Create the Factory
Step 3: Use the Factory

Step 1: Create the classes
Each class has its own pay() method.

```
class CreditCard:
    def pay(self):
        print("Payment made using Credit Card")

class PayPal:
    def pay(self):
        print("Payment made using PayPal")

class BankTransfer:
    def pay(self):
```

```
        print("Payment made using Bank Transfer")
```

Step 2: Create the Factory

The Factory's job is to create the appropriate object.

```
class PaymentFactory:

    @staticmethod
    def create_payment(payment_type):
        if payment_type == "credit_card":
            return CreditCard()

        elif payment_type == "paypal":
            return PayPal()

        elif payment_type == "bank":
            return BankTransfer()

        else:
            raise ValueError("Invalid payment type")
```

Step 3: Use the Factory

```
payment = PaymentFactory.create_payment("paypal")
payment.pay()
```

Example II – Animal Factory

```
class Dog:
    def speak(self):
        print("Woof!")

class Cat:
    def speak(self):
        print("Meow!")

class Cow:
    def speak(self):
        print("Moo!")

#Factory
class AnimalFactory:
    @staticmethod
    def create_animal(animal_type):
        if animal_type == "dog":
            return Dog()
        elif animal_type == "cat":
            return Cat()
        elif animal_type == "cow":
            return Cow()
        else:
            raise ValueError("Unknown animal")

animal = AnimalFactory.create_animal("dog")
animal.speak()
```

1.2. Exercise — Food Ordering System

A restaurant allows customers to order food using different options:

- Pizza
- Burger
- Pasta

Each food item has an prepare() method.

Create a Factory Design Pattern that allows the customer to request a food item without directly creating the corresponding class.

1.3. Factory Method Design Pattern

Instead of one Factory deciding everything, we have different Factory subclasses:

- **Define an interface for creating a factory, but let subclasses decide which class to instantiate.**

1. Abstract Factory / Creator

Defines the Factory Method.

```
class AnimalFactory:
    def create_animal(self):
        pass
```

2. Concrete Factory / Concrete Creator

```
class DogFactory(AnimalFactory):
    def create_animal(self):
        return Dog()

class CatFactory(AnimalFactory):
    def create_animal(self):
        return Cat()
```

3. Abstract Product

Defines the common interface for the products.

```
from abc import ABC, abstractmethod

class Animal(ABC):
    @abstractmethod
    def speak(self):
        pass
```

4. Concrete Product

The actual products.

```
class Dog(Animal):
    def speak(self):
        print("Woof!")


class Cat(Animal):
    def speak(self):
```

```
        print("Meow!")
```

Complete code:

```
from abc import ABC, abstractmethod
# ==========================================
# 1. ABSTRACT PRODUCT
# ==========================================
class Animal(ABC):
    @abstractmethod
    def speak(self):
        pass

# ==========================================
# 2. CONCRETE PRODUCTS
# ==========================================
class Dog(Animal):
    def speak(self):
        print("Woof!")

class Cat(Animal):
    def speak(self):
        print("Meow!")

# ==========================================
# 3. ABSTRACT FACTORY / CREATOR
# ==========================================
class AnimalFactory(ABC):
    @abstractmethod
    def create_animal(self):
        pass

# ==========================================
# 4. CONCRETE FACTORIES
# ==========================================

# Concrete Factory
class DogFactory(AnimalFactory):
    def create_animal(self):
        return Dog()

# Concrete Factory
class CatFactory(AnimalFactory):
    def create_animal(self):
        return Cat()

# ==========================================
# 5. CLIENT
# ==========================================
factory = DogFactory()
animal = factory.create_animal()
animal.speak()
```

1.4. Exercise — Online Notification System

Problem Statement

You are developing a Notification System for an organisation. The system needs to send notifications using different methods:

- Email
- SMS
- Push Notification

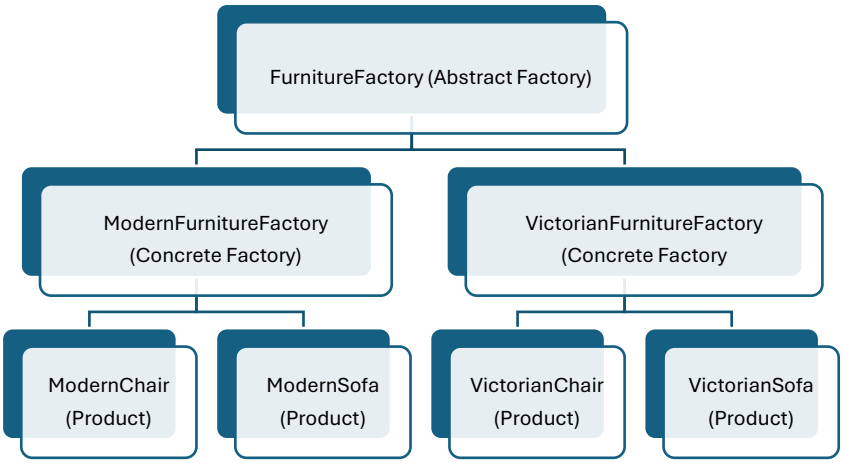
Each notification type should have a send() method.

The system should use the Factory Method Pattern so that:

- EmailFactory creates an Email notification.
- SMSFactory creates an SMS notification.
- PushFactory creates a Push notification.
- The client should not directly create Email(), SMS(), or Push() objects.

1.5. Abstract Factory Pattern

- Factory Method creates one type of product.
- Abstract Factory creates a family of related products.




```
from abc import ABC, abstractmethod
#abstract products
class Chair(ABC):
    @abstractmethod
    def sit(self):
        pass

class Sofa(ABC):
    @abstractmethod
    def relax(self):
        pass

#Concrete Products — Modern family
```

```
class ModernChair(Chair):
    def sit(self):
        print("Sitting on Modern Chair")

class ModernSofa(Sofa):
    def relax(self):
        print("Relaxing on Modern Sofa")

#Concrete Products — Victorian family
class VictorianChair(Chair):
    def sit(self):
        print("Sitting on Victorian Chair")

class VictorianSofa(Sofa):
    def relax(self):
        print("Relaxing on Victorian Sofa")

# Abstract Factory
class FurnitureFactory(ABC):
    @abstractmethod
    def create_chair(self):
        pass

    @abstractmethod
    def create_sofa(self):
        pass

#Concrete Factories
class ModernFurnitureFactory(FurnitureFactory):
    def create_chair(self):
        return ModernChair()

    def create_sofa(self):
        return ModernSofa()

class VictorianFurnitureFactory(FurnitureFactory):
    def create_chair(self):
        return VictorianChair()

    def create_sofa(self):
        return VictorianSofa()

#Client
factory = ModernFurnitureFactory()
chair = factory.create_chair()
sofa = factory.create_sofa()

chair.sit()
sofa.relax()
```

2. Singleton Pattern

The Singleton Design Pattern is used when you want to make sure that:

Only one object (one instance) of a class exists throughout the program.

1. Without Singleton
Normally, every time we use ClassName():

```
class PrinterManager:
    pass

p1 = PrinterManager()
p2 = PrinterManager()
print(p1 is p2) # We have two different objects.
Output: False
```

Imagine an office has **one central printer**. You don't want every employee's program to create a completely new printer manager:

2. Singleton
With Singleton, we want:

```
p1 = PrinterManager()
p2 = PrinterManager()
print(p1 is p2)
Output: True
```

Implementation
One common Python approach is to override __new__():

```
class PrinterManager:

    _instance = None # This variable belongs to the class, not to an
  individual object. there is no object yet

    def __new__(cls):
        if cls._instance is None:
            cls._instance = super().__new__(cls) # create an object
        return cls._instance

p1 = PrinterManager()
p2 = PrinterManager()

print(p1 is p2)
```

Singleton example using Game Settings

```
class GameSettings:

    _instance = None
    def __new__(cls):
        if cls._instance is None:
            cls._instance = super().__new__(cls)

            # Settings
```

```
            cls._instance.sound = 80
            cls._instance.difficulty = "Easy"

        return cls._instance

# Create two settings objects
settings1 = GameSettings()
settings2 = GameSettings()

# Change settings using settings1
settings1.sound = 50
settings1.difficulty = "Hard"

# Access using settings2
print(settings2.sound)
print(settings2.difficulty)

# Check if they are the same object
print(settings1 is settings2)
```

2.1. Exercise — University Configuration Manager

You are developing a university management system where different parts of the application need access to the same university configuration, such as the university name, academic year, and semester. To ensure that all parts of the application use the same configuration, the system should create only one UniversityConfig object. Implement the Singleton Pattern in Python so that whenever a new UniversityConfig object is created, it refers to the same instance. Add methods to set and display the university configuration. Create multiple objects and verify that they are actually the same object using the is operator.

Tasks:

1. Create a UniversityConfig class.
2. Implement Singleton using __new__().
3. Store:
    - University name
    - Academic year
    - Semester
4. Create three objects from UniversityConfig.
5. Set the configuration using one object.
6. Display the configuration using another object.
7. Use is to verify that all objects are the same instance.

3. Builder Pattern

The Builder Pattern is used when we need to create a complex object step by step.

Building a Computer

Imagine you want to buy a computer.
You can choose:

- CPU

- RAM
- Storage
- Graphics card
- Operating system

Instead of putting everything into one complicated constructor:

```
Computer("Intel i7", 16, "1TB SSD", "RTX 4060", "Windows 11")
```

we can build it step by step:

```
computer = ComputerBuilder()
computer.set_cpu("Intel i7")
computer.set_ram(16)
computer.set_storage("1TB SSD")
computer.set_gpu("RTX 4060")
computer.set_os("Windows 11")
my_computer = computer.build()
```

Complete code

```
# ==========================================
# 1. CLIENT
# ==========================================
class Computer:

    def __init__(self, cpu, ram, storage):
        self.cpu = cpu
        self.ram = ram
        self.storage = storage


    def show_details(self):
        print("CPU:", self.cpu)
        print("RAM:", self.ram)
        print("Storage:", self.storage)

# ==========================================
# 2. BUILDER / CONCRETE BUILDER
# ==========================================
class ComputerBuilder:

    def __init__(self):
        self.cpu = None
        self.ram = None
        self.storage = None


    def set_cpu(self, cpu):
        self.cpu = cpu
        return self

    def set_ram(self, ram):
        self.ram = ram
        return self

    def set_storage(self, storage):
```

```
        self.storage = storage
        return self

    def build(self):
        return Computer(
            self.cpu,
            self.ram,
            self.storage
        )


# ========================================
# 3. DIRECTOR
# ========================================
class ComputerDirector:

    def build_basic_computer(self, builder):
        builder.set_cpu("Intel i5")
        builder.set_ram("8 GB")
        builder.set_storage("512 GB SSD")
        return builder.build()

builder= ComputerBuilder()
director = ComputerDirector()
computer = director.build_basic_computer(builder)
computer.show_details()


'''
computer = (
    ComputerBuilder()
    .set_cpu("Intel i7")
    .set_ram("16 GB")
    .set_storage("1 TB SSD")
    .build()
)
'''
```

```
class Pizza:

    def __init__(self, size, crust, cheese, toppings, sauce):
        self.size =size
        self.crust=crust
        self.cheese=cheese
        self.toppings=toppings
        self.sauce=sauce


    def show_pizza(self):
        print("Size: ", self.size)
        print("Crust: ", self.crust)
```

```
        print("Cheese: ", self.cheese)
        print("Toppings: ", self.toppings)
        print("Sauce: ", self.sauce)


class PizzaBuilder:
    def __init__(self):
        self.size = None
        self.crust= None
        self.cheese= None
        self.toppings= []
        self.sauce= None


    def set_size(self, size):
        self.size = size
        return self


    def set_crust(self, crust):
        self.crust = crust
        return self


    def add_cheese(self, cheese):
        self.cheese = cheese
        return self


    def add_toppings(self, toppings):
        self.toppings = toppings
        return self


    def set_sauce(self, sauce):
        self.sauce = sauce
        return self


    def build(self):
         return Pizza(
            self.size,
            self.crust,
            self.cheese,
            self.toppings,
            self.sauce
         )

pizza = (
    PizzaBuilder()
    .set_size("Large")
    .set_crust("Thin")
    .add_cheese("Smoked")
    .add_toppings("Mushroom")
    .set_sauce("BBQ")
    .build())

pizza.show_pizza()
```

3.1. Exercise — Build a Travel Package

Problem Statement

A travel company allows customers to customise a travel package.

A package can include:

- **Destination:** Auckland, Sydney, Melbourne, etc.
- **Hotel:** 3-star, 4-star, 5-star
- **Transport:** Flight, Train, Bus
- **Meal plan:** None, Breakfast, Full Board
- **Activities:** City Tour, Museum, Adventure Tour
- **Insurance:** Yes/No

Your task is to use the **Builder Pattern** to construct a travel package step by step.

STRUCTURAL PATTERNS

1. Decorator Pattern

The Decorator Pattern is a structural design pattern used to add new functionality to an existing object without changing its original class.

Basic Decorator

```
class Coffee:
    def cost(self):
        return 5
```

A customer can add:

- Milk → +$1
- Sugar → +$0.50
- Whipped cream → +$2

Instead of creating many classes such as:

CoffeeWithMilk
CoffeeWithMilkAndSugar
CoffeeWithMilkSugarAndCream
...
we can decorate the coffee.

```
# ========================================
# 1. PRODUCT
# ========================================
class Coffee:
    def cost(self):
        return 5

    def description(self):
        return "Coffee"

# ========================================
```

```
# 2. BASE DECORATOR
# ========================================
class CoffeeDecorator:
    def __init__ (self, coffee):
        self.coffee = coffee

    def cost(self):
        return self.coffee.cost()

# ========================================
# 3. CONCRETE DECORATORS
# ========================================
class MilkDecorator(CoffeeDecorator):

    def cost(self):
        return self.coffee.cost() + 1

class SugarDecorator(CoffeeDecorator):
    def cost(self):
        return self.coffee.cost() + 0.5

class CreamDecorator(CoffeeDecorator):
    def cost(self):
        return self.coffee.cost() + 0.5

# ========================================
# 4. CLIENT - CREATE COFFEE
# ========================================
'''
coffee = Coffee()
coffee = MilkDecorator(coffee)
coffee = SugarDecorator(coffee)
coffee = CreamDecorator(coffee)
'''

coffee = CreamDecorator(SugarDecorator(MilkDecorator(Coffee())))
print(coffee.cost())
```

1.1 Exercise — Car Customisation

Problem Statement

A car company has a basic car:

Basic Car
Price: $25,000
Customers can optionally add:

- GPS → +$500
- Sunroof → +$1,000
- Leather Seats → +$1,500
- Premium Sound System → +$800

The company does **not** want to create separate classes such as:

CarWithGPS

CarWithGPSAndSunroof
CarWithGPSSunroofAndLeather
CarWithGPSSunroofLeatherSound
...

Instead, use the **Decorator Pattern** so that customers can add features dynamically.

2. Adapter Pattern

The Adapter Pattern allows two classes to work together when their interfaces are incompatible.

2.1 Combination

Combination means the Adapter contains an object of the Adaptee.

```
# Target
class Target:

    def request(self):
        pass

# Adaptee
class Adaptee:

    def specific_request(self):
        print("Adaptee: specific request")

# Adapter
class Adapter(Target):

    def __init__(self, adaptee):
        self.adaptee = adaptee


    def request(self):
        self.adaptee.specific_request()

# Client
adaptee = Adaptee()
adapter = Adapter(adaptee)
adapter.request()
```

The client doesn't need to know that the Adaptee uses specific_request().

Two Ways to Implement Adapter

1. **Combination / Composition**
   The Adapter has an Adaptee object. This is often called Object Adapter.

```
class Adapter:

    def __init__(self, adaptee):
        self.adaptee = adaptee
```

Complete Code

```
# Target
class Target:
```

```
    def request(self):
        pass

# Adaptee
class Adaptee:

    def specific_request(self):
        print("Adaptee: specific request")

# Adapter
class Adapter(Target):

    def __init__(self, adaptee):
        self.adaptee = adaptee

    def request(self):
        self.adaptee.specific_request()

# Client
adaptee = Adaptee()
adapter = Adapter(adaptee)
adapter.request()
```

2.2 Multiple Inheritance

The Adapter inherits from both Target and Adaptee.

```
class Adapter(Target, Adaptee):

    def request(self):
        self.specific_request()
```

Complete Example

```
class Target:
    def request(self):
        pass

class Adaptee:
    def specific_request(self):
        print("Adaptee: specific request")

class Adapter(Target, Adaptee):

    def request(self):
        self.specific_request()

adapter = Adapter()
adapter.request()
```

2.3 Exercise — Payment System

You are developing an online shopping application that supports different payment systems. The application expects every payment service to provide a pay(amount) method. However, an existing third-party payment system provides a make_payment(amount) method instead. Since the third-party system cannot be modified, you need to create an Adapter that allows the existing payment system to work with the application's expected interface. Implement the Adapter

Pattern using Combination (Composition), where the Adapter contains an object of the existing payment system. As an extension, implement the same solution using Multiple Inheritance. The final program should allow the client to call pay(500) and produce the message "Payment of $500 made using Old Payment System." Identify the Target, Adaptee, and Adapter classes in your implementation.

BEHAVIORAL PATTERNS

1. Observer Pattern

- The Observer Pattern is used when one object changes and several other objects need to know about that change.
- One object (Subject) keeps a list of other objects (Observers) and automatically notifies them when something changes.

Example - When the weather changes, several people want to know.

```
class Observer:

    def update(self, temperature):
        pass

# Concrete Observer
class Person(Observer):

    def __init__(self, name):
        self.name = name
    def update(self, temperature):
        print(self.name, "knows temperature is", temperature)

# Subject
class WeatherStation:

    def __init__(self):
        self.observers = []

    def add_observer(self, observer):
        self.observers.append(observer)


    def notify(self, temperature):
        for observer in self.observers:
            observer.update(temperature)

# Create observers
person1 = Person("Ali")
person2 = Person("Sara")

# Create subject
weather = WeatherStation()

# Subscribe people
weather.add_observer(person1)
weather.add_observer(person2)

# Temperature changes
```

```
weather.notify(25)
```

Example - YouTube channel.

```
# Observer
# The Observer defines what an observer must do when it receives an update.
class Observer:

    def update(self):
        pass

# Concrete Observer
# The Concrete Observer is the actual object that receives the
notification.
class Student(Observer):

    def __init__(self, name):
        self.name = name
    def update(self):
        print(self.name, "received a notification")

# Subject
# The Subject is the object whose state can change.
class Subject:

    def __init__(self):
        self.observers = []


    def attach(self, observer):
        self.observers.append(observer)


    def notify(self):
        for observer in self.observers:
            observer.update()


# Concrete Subject
# The Concrete Subject is the actual subject that changes its state.
class YouTubeChannel(Subject):

    def upload_video(self, title):
        print("New video uploaded:", title)
        self.notify()


# Create Subject
channel = YouTubeChannel()

# Create Observers
student1 = Student("Ali")
student2 = Student("Sara")
student3 = Student("John")

# Subscribe students
channel.attach(student1)
channel.attach(student2)
channel.attach(student3)
```

```
# Upload video
channel.upload_video("Python Design Patterns")
```

1.1 Exercise — Stock Price Monitor

Problem Statement

You are developing a **Stock Price Monitoring System**. A company has a stock price. Whenever the stock price changes, all registered investors should automatically receive the new price.

Task 1 – Create the Observer

Task 2 – Create the Concrete Observer

- Create an Investor class.
- Each investor should have a name.
- When the investor receives an update, display:
- Ali received new stock price: 105

Task 3 – Create the Subject

Create a Stock class that:

- Maintains a list of investors
- Allows investors to subscribe
- Notifies all investors

Task 4 – Change the Stock Price